

Relazione Progetto: File System Statistics (FSStat)

Programmazione Concorrente e Distribuita

Il tuo Nome e Cognome

Anno Accademico 2025/2026

Sommario

Il presente documento descrive la progettazione e l'implementazione della libreria **FSStatLib**, sviluppata in Java 25. L'obiettivo del progetto è analizzare un albero di directory del file system, calcolando il numero totale di file e la loro distribuzione in fasce di dimensione. L'elaborazione è stata parallelizzata utilizzando in modo esclusivo i Virtual Threads (Project Loom), per garantire alta scalabilità e reattività.

Indice

1	Introduzione e Requisiti	2
2	Analisi del Problema e Scelte Concorrenti	2
2.1	Approccio Thread-Per-Task con i Virtual Threads	2
2.2	Evitare il problema dei Pinned Threads	2
3	Architettura e Design	2
3.1	Meccanismo di Cancellazione	3
4	Risultati e Osservazioni	3

1 Introduzione e Requisiti

Il progetto prevede la creazione di una libreria asincrona in grado di esplorare il file system a partire da una directory radice e fornire un report statistico (numero totale di file e raggruppamento in bande in base alla dimensione).

Oltre al requisito base, è stato implementato il requisito opzionale (punto di eccellenza) che prevede la possibilità di fermare la computazione in qualsiasi momento e di ricevere aggiornamenti in tempo reale sulle statistiche.

2 Analisi del Problema e Scelte Concorrenti

L'attraversamento ricorsivo di un file system e l'interrogazione delle proprietà dei file sono tipiche operazioni legate all'I/O. In questi scenari, il modello di programmazione a thread fisici ("platform threads") presenta gravi limiti di scalabilità a causa dell'elevato consumo di memoria allocato per lo stack di ogni thread Virtual Threads.pdf].

2.1 Approccio Thread-Per-Task con i Virtual Threads

Per risolvere questo problema, è stata adottata l'API dei **Virtual Threads**. I Virtual Threads condividono caratteristiche con la memoria virtuale, in quanto sono economici e possono esistere in grande quantità, appoggiandosi su un numero ristretto di *carrier threads* del sistema operativo Virtual Threads.pdf].

La libreria utilizza il pattern *Thread-per-task* Virtual Threads.pdf]: per ogni sotto-directory incontrata durante l'esplorazione, viene sottomesso un nuovo task a un `VirtualThreadPoolExecutor` dedicato. Quando un thread esegue un'operazione bloccante (ad esempio la lettura del contenuto della cartella o la `sleep()` per rallentare l'interfaccia), esso effettua automaticamente l'*unmounting* dal carrier thread, rilasciandolo per elaborare altri Virtual Threads e garantendo così un throughput altissimo Virtual Threads.pdf].

2.2 Evitare il problema dei Pinned Threads

L'implementazione dell'aggiornamento in tempo reale sulla GUI richiede che tutti i task condividano lo stesso oggetto `FSReport`. Tuttavia, l'uso di blocchi `synchronized` o metodi nativi in Java comporta il blocco sia del Virtual Thread sia del thread di piattaforma ad esso associato (fenomeno noto come *Pinned Thread*), annullando i benefici di scalabilità Virtual Threads.pdf].

Per ovviare a questo problema, i monitor per lo stato condiviso sono stati implementati usando la classe `ReentrantLock` Virtual Threads.pdf], la quale è stata nativamente adattata nel JDK per supportare il corretto *unmounting* dei Virtual Threads.

3 Architettura e Design

Il progetto è suddiviso seguendo il principio di Separazione delle Responsabilità (Separation of Concerns). I package principali sono tre:

- **it.unibo.fsstat.lib**: Contiene il dominio e la logica concorrente. Non ha dipendenze da componenti grafiche o di I/O su console.
- **it.unibo.fsstat.cli**: Contiene un programma a riga di comando minimale (metodo `main` standard).
- **it.unibo.fsstat.gui**: Implementa un'interfaccia grafica in Java Swing che permette all'utente di selezionare la directory, far partire il task, interromperlo in modo pulito e visualizzare le statistiche man mano che si aggiornano.

3.1 Meccanismo di Cancellazione

La possibilità di interrompere dinamicamente il thread è stata gestita introducendo un oggetto `TaskController` condiviso con flag volatili. Durante l'iterazione sulle cartelle, i thread controllano periodicamente questo stato; se viene rilevata la richiesta di arresto, il ciclo si interrompe e propaga la cancellazione ai `Future` dei *sub-tasks*.

4 Risultati e Osservazioni

Dai test effettuati localmente, l'overhead della creazione di migliaia di Virtual Threads è risultato impercettibile, permettendo all'applicazione di consumare una frazione della RAM che sarebbe stata necessaria usando un pool cached di platform threads [Virtual Threads.pdf]. Il sistema è risultato stabile, senza mai incorrere in `OutOfMemoryError`, confermando le tesi sulla scalabilità di questo nuovo paradigma su workload I/O-bound.